

Plan de servidor y metajuego — Juego de carreras arcade con Nakama

Oct 4, 2026 · @Jairo

Resumen

Con login, rooms y carrera funcionando, quedan ocho bloques de servidor: resultados de carrera confiables, matchmaking, garaje y economía, ranked con temporadas y torneos, social, misiones y pase, tienda multiplataforma y operación en producción. Este documento explica cómo funciona cada sistema y cómo se conecta con los demás. Las tareas de cada fase, con casillas para seguir el avance, están en

[Checklist de desarrollo por fases — Juego de carreras con Nakama](#) .

La pieza que sostiene todo lo demás es el **pipeline de resultados de carrera** (sección Carrera). Hoy el cliente es la única fuente de verdad. Antes de dar monedas, rango o premios, el servidor necesita un único punto por donde entra cada resultado. Por eso es la Fase 1, y por eso la leaderboard se rediseña para colgar de ahí.

Estado	Sistema
Hecho	Nakama en Docker, conexión desde Unity, login, creación de rooms, carrera por relay
Se reconstruye	Leaderboard: la actual, escrita por el cliente, se descarta y se rehace escrita solo por el servidor
Por hacer	Todo lo demás de este documento

Decisiones y supuestos

- **Red:** relay puro. El servidor no simula física ni ve los mensajes de la carrera. El diseño deja preparado el paso a validación sin reescribir el metajuego.
- **Lógica de servidor:** TypeScript sobre el runtime de Nakama, un módulo por sistema. Todo el metajuego se construye en casa, sin librerías comerciales.
- **Carrera:** de 2, 4 o 6 autos. En carrera rápida los bots completan la parrilla, simulados por el host.
- **Contenido de lanzamiento:** al menos 6 pistas y 6 autos.
- **Plataformas:** móviles y PC, con una sola cuenta por jugador y una sola pool de matchmaking por región.

- **Regiones:** América y Europa. Un servidor principal para cuentas y metajuego, y un nodo de relay por región para cola y carrera.
- **Negocio:** venta con dinero real de paquetes de monedas y gemas, skins y colores, autos, personalizaciones, pase de temporada y paquetes de evento.
- **Ranked:** estadísticas igualadas al tope de la clase.
- **Equipo:** una persona en servidor y una en cliente. Se prioriza lo que Nakama ya trae (wallet, storage, leaderboards, tournaments, groups, chat, notifications) antes que construir sistemas propios.

Principios de diseño

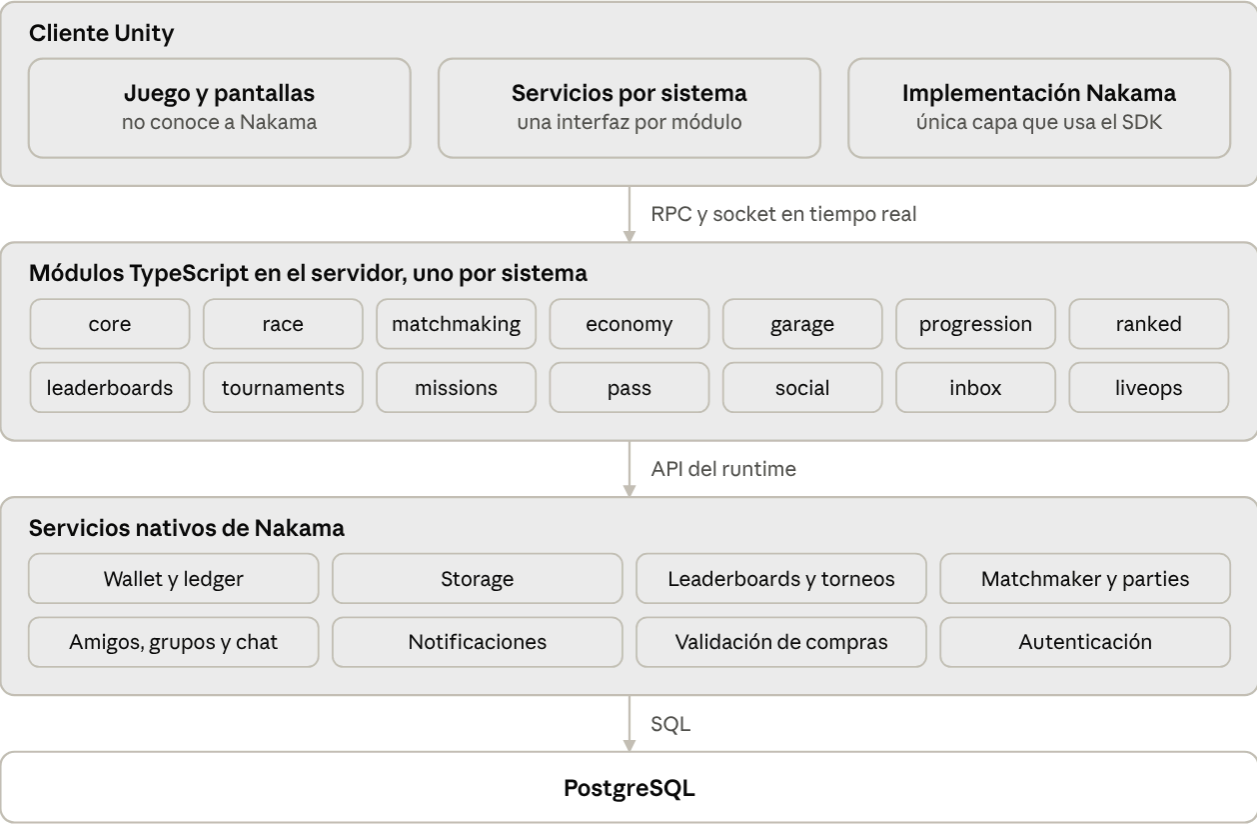
Relay puro significa que Nakama reenvía los mensajes de la carrera sin inspeccionarlos y solo conoce el ID del match y quién está presente. Todo lo que el servidor sepa de una carrera se lo tendrá que contar un cliente. Estas siete reglas hacen que eso sea manejable hoy y reemplazable mañana.

1. **El cliente nunca escribe estado persistente.** Monedas, autos, rango, progreso de misiones y registros de leaderboard solo cambian dentro de un RPC. Los objetos de storage van con permiso de escritura 0.
2. **Un solo punto de entrada para resultados.** `race_submit_result` es la única puerta. De ahí sale un evento interno `RaceCompleted` y todos los sistemas (premios, XP, rating, leaderboards, misiones, pase, clubes) reaccionan a él.
3. **Cada resultado lleva un nivel de confianza.** `client`, `quorum` o `server`. Hoy se usan los dos primeros. Cuando haya validación, solo cambia quién produce el resultado; economía y ranking no se tocan.
4. **Todo RPC que entrega algo es idempotente.** La clave es el ID de sesión de carrera, de transacción o de reclamo. Un reintento por mala red nunca paga dos veces.
5. **Los datos de diseño viven en catálogos versionados en el servidor.** Precios, premios, misiones y niveles del pase se cambian sin publicar un build.
6. **Primero lo nativo de Nakama.** Wallet con ledger, storage, leaderboards, tournaments, groups, chat, notifications, validación de compras.
7. **Reinicios perezosos.** Lo diario y lo semanal se calcula al leer, comparando sellos de fecha en UTC. Los únicos procesos programados son los resets de leaderboards y torneos, que Nakama ya dispara con sus callbacks.

Arquitectura del servidor

Un módulo TypeScript por sistema, todos registrados en `InitModule` , que se comunican solo a través del evento `RaceCompleted` y de funciones públicas de cada módulo. Ningún módulo lee el storage de otro directamente.

El cliente solo ve RPC; los módulos solo usan lo nativo de Nakama



arquitectura · 4 capas, 14 módulos

Módulos

Módulo	Responsabilidad	RPCs principales	Se apoya en
core	Catálogos, configuración, idempotencia, errores, versión mínima de cliente	config_get	Storage

Módulo	Responsabilidad	RPCs principales	Se apoya en
race	Sesiones de carrera, recepción de resultados, emisión de RaceCompleted	race_session_create , race_session_join , race_session_start , race_session_get , race_submit_result	Storage
matchmaking	Hook de emparejamiento, selección de host, relleno con bots	— (hook matchmakerMatched)	Matchmaker, Parties
economy	Monedas, premios, tienda, compras	store_get , store_buy , iap_validate	Wallet, validación de compras
garage	Autos, mejoras, cosméticos, loadout activo	garage_get , car_buy , car_upgrade , loadout_set	Storage, Wallet
progression	XP, nivel de jugador, desbloques	profile_get	Storage
ranked	Rating, divisiones, temporadas	ranked_get	Storage, Leaderboards
leaderboards	Registro de tablas, escritura, consultas compuestas	lb_get	Leaderboards
tournaments	Eventos con premio	tournament_list , tournament_join	Tournaments
missions y pass	Misiones diarias y semanales, pase de temporada	missions_get , mission_claim , pass_get , pass_claim	Storage, Wallet
social	Clubes, moderación de chat, reportes	club_get , report_player	Friends, Groups, Chat
inbox	Correo con premios adjuntos	inbox_list , inbox_claim	Notifications
liveops	Flags, calendario de eventos, mantenimiento	— (se sirve por config_get)	Storage

Almacenamiento

Todas las colecciones van con escritura 0 (solo servidor). Las monedas no van en storage: viven en el wallet de Nakama como `coins` y `gems`, con su ledger.

Colección / clave	Dueño	Lectura	Contenido
<code>profile/main</code>	Jugador	Pública	Nivel, XP, avatar, auto mostrado, club
<code>garage/cars</code>	Jugador	Dueño	Autos, niveles de mejora, cosméticos
<code>garage/loadout</code>	Jugador	Pública	Auto y configuración activos
<code>ranked/s{N}</code>	Jugador	Pública	Rating, división, carreras jugadas en la temporada N
<code>missions/daily</code> , <code>missions/weekly</code>	Jugador	Dueño	Misiones asignadas, progreso, sello de fecha
<code>pass/s{N}</code>	Jugador	Dueño	XP de pase, niveles reclamados, premium sí/no
<code>race_sessions/{sessionId}</code>	Sistema	Solo por RPC	Roster, pista, modo, loadouts, host, estado, resultados
<code>catalogs/{nombre}</code>	Sistema	Solo por RPC	Autos, pistas, tienda, misiones, pase, temporadas
<code>liveops/config</code>	Sistema	Solo por RPC	Flags, eventos activos, versión mínima

Cada objeto lleva un campo `schemaVersion`. Cuando cambie la forma de un dato, el módulo dueño lo migra al leerlo.

Catálogos

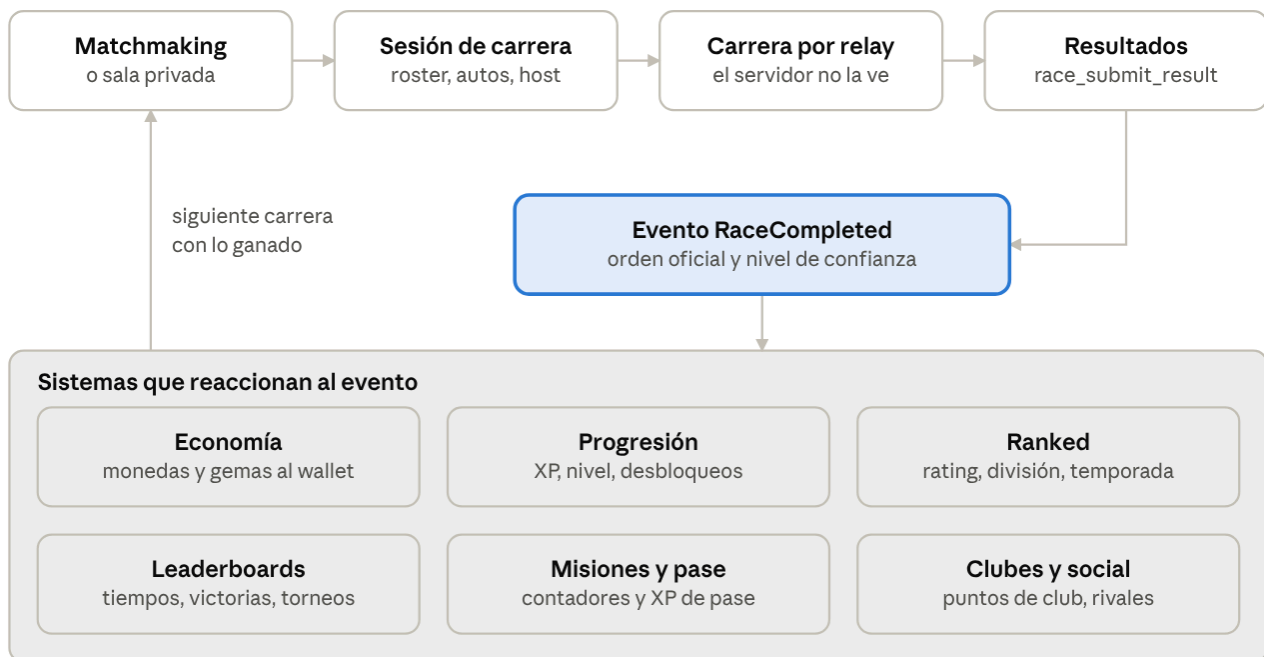
Los catálogos se escriben como JSON en el repositorio del servidor y se cargan al arrancar. `config_get` devuelve al cliente los catálogos públicos con un hash; el cliente solo vuelve a descargar si el hash cambió. El catálogo de pistas incluye un **tiempo mínimo plausible** por pista y por clase de auto, que usa la validación de resultados.

Lado Unity

Cada módulo de servidor tiene un servicio espejo en el cliente detrás de una interfaz (`IRaceSessionService` , `IEconomyService` , `IGarageService` ...), con la implementación Nakama en la capa de infraestructura. El juego nunca llama al SDK de Nakama fuera de esas implementaciones.

Cómo encaja todo

Todo el metajuego cuelga de un solo evento: **RaceCompleted**



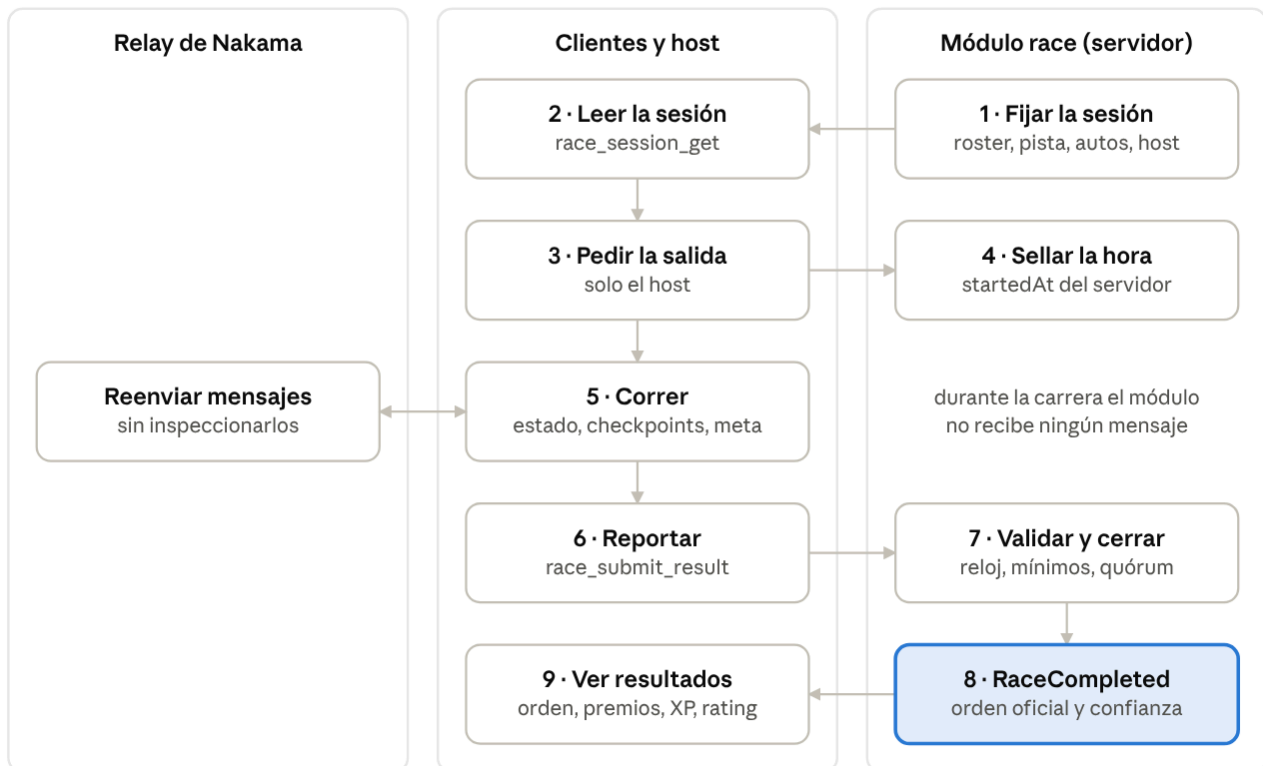
bucle de juego · 4 pasos, 1 evento, 6 sistemas

Cada carrera entra al servidor por un único RPC. El evento que sale de ahí reparte a seis sistemas, y lo que el jugador gana en ellos (monedas, autos, rango) es con lo que vuelve a la siguiente carrera.

Carrera

La carrera sigue corriendo por relay como hoy; lo nuevo es una **sesión de carrera** en el servidor que la envuelve. La sesión le da al servidor un roster, un reloj y un lugar donde recibir resultados, sin ver un solo mensaje de gameplay.

El servidor envuelve la carrera sin verla: nueve pasos, tres actores



ciclo de una carrera · 9 pasos, 3 actores

Ciclo de una sesión

- Crear.** En carreras emparejadas se crea el hook `matchmakerMatched`, que ya conoce a los participantes. En salas privadas el match se sigue creando desde el cliente, como hoy, y el host registra la sesión con `race_session_create`.
- Fijar.** El servidor guarda en la sesión el roster, la pista, el modo, el host, el orden de sucesión de host y el loadout de cada jugador leído de `garage/loadout`. Los clientes aplican las estadísticas de los rivales desde la sesión, no desde lo que diga cada rival.
- Arrancar.** El host marca el inicio. El servidor sella `startedAt` con su propio reloj. La cuenta regresiva se sincroniza contra la hora del servidor.
- Correr.** Relay puro, sin cambios.
- Reportar.** Cada cliente llama `race_submit_result` al cruzar la meta: tiempo total, tiempos por vuelta, parciales por checkpoint y el orden final que vio. El host reporta además los bots.
- Cerrar.** La sesión cierra cuando todos reportaron o al vencer el plazo (30 s después del primer reporte). Quien no reportó queda como abandono.
- Emitir.** El servidor calcula el orden oficial, asigna el nivel de confianza y emite `RaceCompleted`. La respuesta del RPC ya trae premios, XP y cambio de rating para la pantalla de resultados.

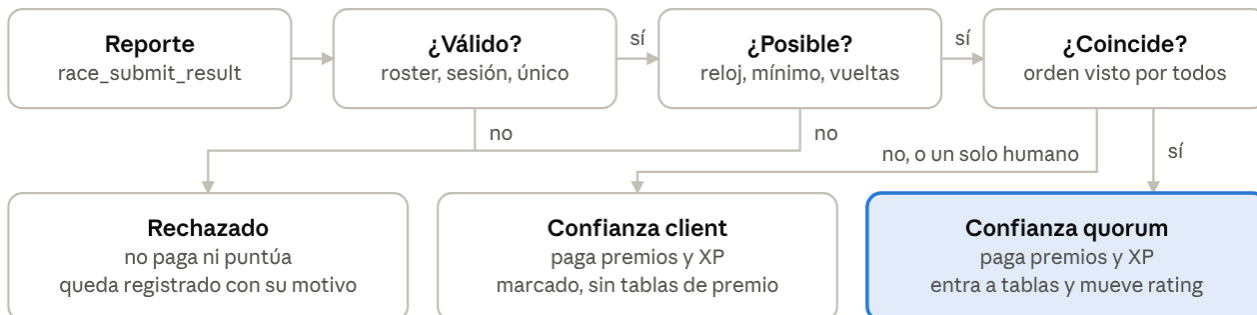
Qué valida el servidor hoy

Sin ver la carrera, el servidor puede rechazar lo imposible y marcar lo sospechoso:

- El que reporta está en el roster, la sesión está abierta y no había reportado.
- El tiempo reportado no supera el tiempo real transcurrido desde `startedAt` según el reloj del servidor.
- El tiempo total no baja del mínimo plausible de la pista para esa clase de auto.
- La suma de vueltas coincide con el total y el número de vueltas con el modo.
- El orden final coincide entre la mayoría de los reportes. Si coincide, confianza `quorum`; si hay un solo humano o hay desacuerdo, `client` y el resultado se marca para revisión.

Un resultado rechazado no paga ni puntúa. Uno marcado paga normal, pero no entra a leaderboards de premio hasta revisarse.

Un resultado se rechaza, se marca o se acepta según tres comprobaciones



validación de resultados · 3 comprobaciones, 3 salidas

Reconexión y abandono

- La sesión guarda el ID del match. Al reabrir el juego, `race_session_get` dice si hay una carrera viva y el cliente se reincorpora.
- Durante la desconexión, el auto del jugador lo conduce el autodrive en el host durante 20 s de gracia. Pasado ese tiempo es abandono.
- Si cae el host, toma el control el siguiente en el orden de sucesión guardado en la sesión.
- En ranked el abandono cuenta como último lugar. Tres abandonos en 24 h bloquean la cola ranked 15 minutos.

Bots

Los simula el host con la IA del sistema de conducción. No tienen cuenta, no reciben premios y no cuentan para el rating: el rating se calcula solo entre humanos. Los premios de una carrera con bots usan la posición real, bots incluidos, para que ganarle a la parrilla siga valiendo.

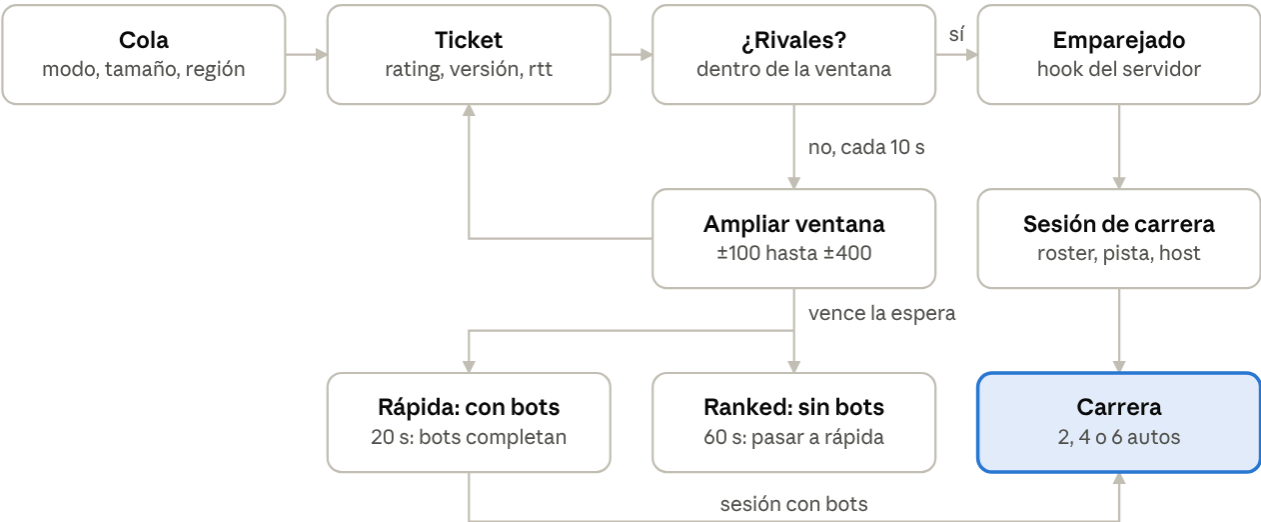
Modos

Modo	Autos	Emparejamiento	Rating	Premios	Leaderboards
Carrera rápida	2, 4 o 6 a elección; bots completan	Matchmaker	No	Completo	Victorias semanales
Ranked	2, 4 o 6 según humanos en cola; sin bots	Matchmaker por rating	Sí	Completo + temporada	Ranked de temporada
Sala privada	2, 4 o 6, elige el host	Código o invitación	No	Reducidos	Ninguna
Contrarreloj	1	Sin rivales	No	Por superar marca	Tiempo por pista
Torneo	Según evento	Según evento	Según evento	Del evento	Tabla del torneo

Matchmaking

Se usa el matchmaker de Nakama para formar carreras de 2, 4 o 6 autos; las salas actuales quedan como modo privado. El hook `matchmakerMatched` crea la sesión de carrera y deja el match como relay.

La cola amplía la búsqueda y nunca deja al jugador esperando



flujo de matchmaking · 1 bucle, 3 salidas

Propiedades del ticket

Propiedad	Valor	Uso en la consulta
mode	quick , ranked , id de torneo	Obligatoria, coincidencia exacta
version	Versión de protocolo del cliente	Obligatoria, coincidencia exacta
region	america , europe	Obligatoria; cada región tiene su nodo de relay
size	2, 4 o 6	Coincidencia exacta en carrera rápida; no se usa en ranked
rating	Rating de temporada	Rango en ranked; preferencia en rápida
class	Clase del auto equipado	Coincidencia exacta
platform	android , ios , pc	Ignorada hoy; filtro cuando se active la segmentación
input	touch , gamepad , keyboard , wheel	Ignorada hoy; candidata a segmentar antes que la plataforma
rtt	Latencia medida al nodo de relay	Para elegir host

Reglas

- **Tamaños.** En carrera rápida el jugador elige 2, 4 o 6 autos. En ranked hay una sola cola: el matchmaker forma carreras de 2 a 6 humanos en múltiplos de 2 y prefiere la más grande que pueda llenar.
- **Ventana de rating que se abre.** El ticket ranked empieza en ± 100 de rating. Cada 10 s sin emparejar, el cliente lo cancela y lo reenvía con ± 100 más, hasta ± 400 .
- **Bots.** En carrera rápida, a los 20 s se arranca con los humanos que haya y bots hasta completar el tamaño elegido. En ranked no hay bots; a los 60 s sin emparejar se ofrece carrera rápida.
- **Host.** El hook elige como host al jugador con menor `rtt` y guarda el resto ordenado como sucesión.
- **Grupos.** Los amigos entran juntos a la cola con Parties de Nakama. El rating del grupo para emparejar es el más alto de sus miembros.
- **Versión y región.** Dos clientes con distinta versión de protocolo o de distinta región nunca se emparejan. `config_get` avisa cuando hay que actualizar.

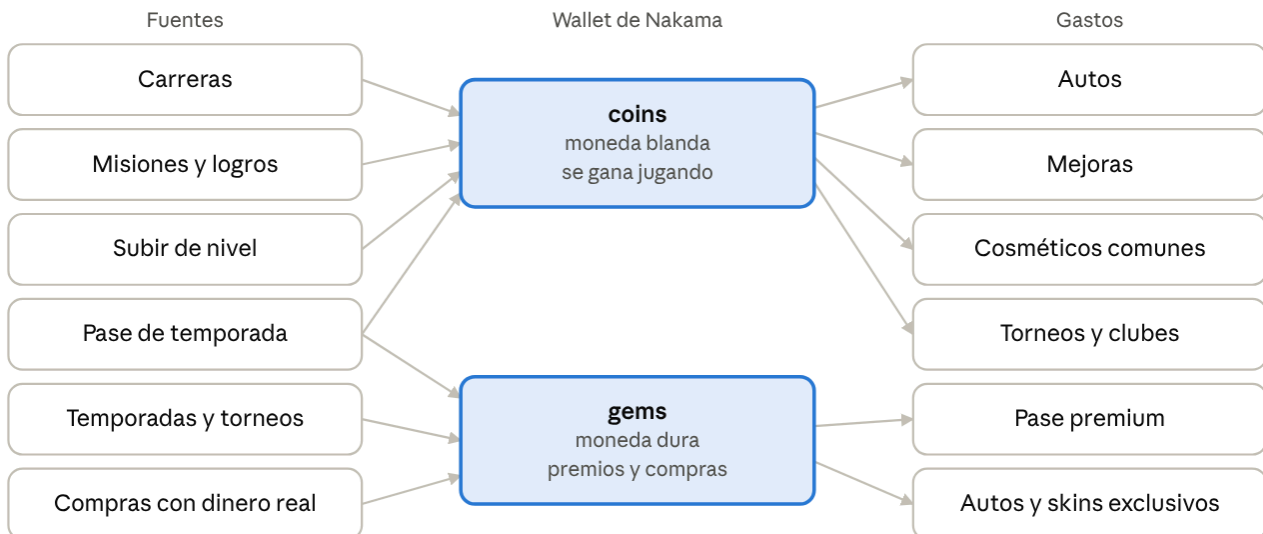
Segmentación por plataforma

Dentro de cada región la pool es única al lanzar. `liveops/config` lleva `mm.segmentBy` con valores `none`, `input` o `platform`; el cliente arma la consulta del ticket según ese valor. Activar la segmentación es cambiar una línea de configuración, sin build. Conviene medir primero la diferencia de resultados entre táctil y gamepad: en carreras suele pesar más el control que la plataforma.

Garaje, progresión y economía

Dos monedas, autos agrupados por clase, mejoras con tope por clase y cosméticos sin efecto en pista. Los números de esta sección son valores de partida para ajustar con datos reales.

Todo lo que entra y sale del wallet pasa por dos monedas



economía · 6 fuentes, 2 monedas, 6 gastos

Monedas

Moneda	De dónde sale	En qué se gasta
coins (blanda)	Carreras, misiones, pase gratuito, subir de nivel	Autos, mejoras, cosméticos comunes
gems (dura)	Pase, premios de temporada y torneo, compras	Pase premium, cosméticos exclusivos, autos especiales

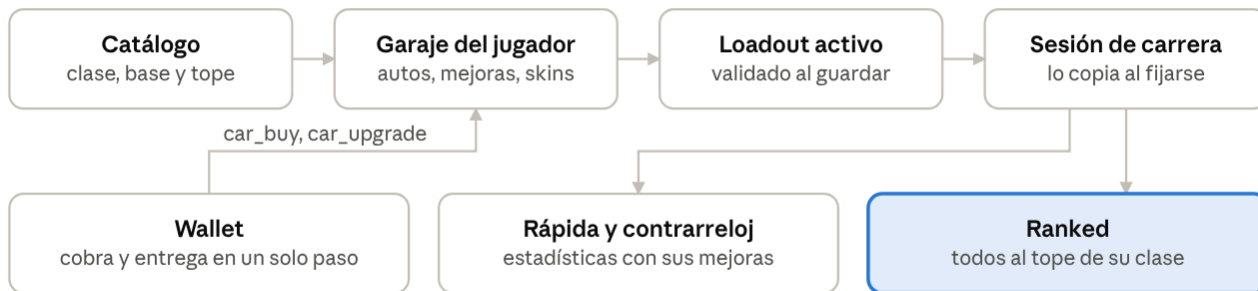
Cada movimiento del wallet lleva en el ledger el motivo y el ID que lo originó (`race: {sessionId}` , `mission:{id}` , `store:{offerId}`). Eso permite auditar a un jugador y medir cuánto entra y cuánto sale por fuente.

Premios por carrera

Premio = base por posición × multiplicador de modo + bonos. Base para 6 autos: 100, 80, 65, 50, 40, 30 monedas; para 4 autos: 100, 70, 50, 35; para 2 autos: 100 y 50. Multiplicadores: rápida ×1, ranked ×1,25, privada ×0,25. Bonos: primera victoria del día +100; carrera completada sin abandonar +10. La XP sigue la misma tabla a la mitad, con un piso de 20.

Autos y mejoras

El servidor decide con qué auto y qué estadísticas corre cada jugador



del catálogo a la carrera · 4 pasos, 2 reglas de estadísticas

- **Clases.** Cada auto pertenece a una clase (D, C, B, A, S) según su rendimiento base. La clase define contra quién compite.
- **Mejoras.** Cuatro líneas por auto (motor, neumáticos, nitro, manejo) con 5 niveles cada una y costo creciente. Un auto mejorado al máximo llega al tope de su clase, nunca lo supera.
- **Cosméticos.** Pintura, llantas, calcomanías, estela y bocina. No cambian estadísticas.
- **Loadout.** `loadout_set` valida que el jugador posea el auto y las piezas, y escribe `garage/loadout`. Es lo único que la sesión de carrera lee.

En ranked las estadísticas se igualan al tope de la clase para todos (decisión tomada). Las mejoras importan en carrera rápida y contrarreloj; el ranked mide manejo. Así el rango no se puede comprar y la validación de resultados es más simple.

Progresión del jugador

Nivel 1 a 50 por XP. Cada nivel entrega monedas; ciertos niveles desbloquean contenido: nivel 3 misiones diarias, nivel 5 ranked, nivel 8 clubes, nivel 10 torneos, y cada clase de auto en la tienda a los niveles 1, 6, 12, 20 y 30. Los desbloqueos viven en el catálogo.

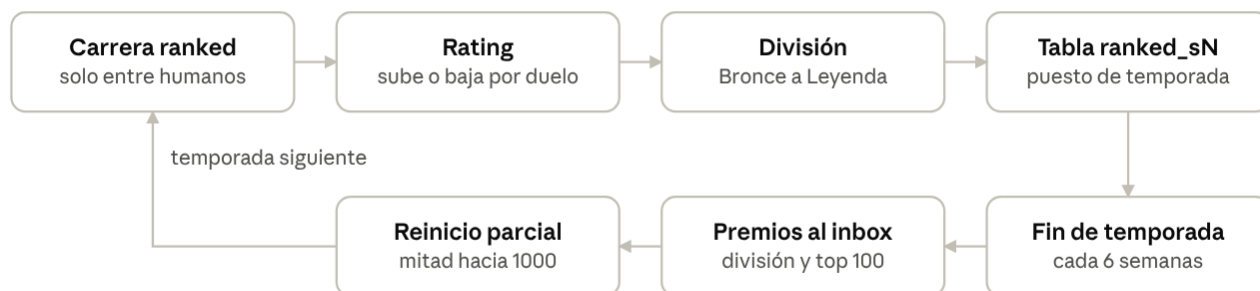
Reglas de implementación

- Comprar o mejorar descuenta del wallet y escribe el garaje en una sola transacción (`multiUpdate`). Si una parte falla, no ocurre ninguna.
- El cliente manda solo el ID de lo que quiere. Precio, requisitos y resultado los resuelve el servidor desde el catálogo.
- Antes de fijar precios, modelar en una hoja de cálculo cuántas carreras cuesta cada auto y cada mejora. Objetivo inicial: primer auto nuevo en 2 a 3 días de juego normal.

Ranked, temporadas y torneos

El ranked usa un rating tipo Elo adaptado a carreras de varios jugadores, agrupado en seis divisiones y reiniciado parcialmente cada temporada de 6 semanas. Los torneos usan el sistema de tournaments de Nakama.

Cada temporada el rating sube con las carreras y vuelve a acercarse a 1000



ciclo de temporada ranked · 7 pasos

Rating

Cada carrera se trata como un duelo contra cada uno de los otros humanos. El cambio de rating del jugador i en una carrera con N humanos es:

$$\Delta_i = \frac{K}{N-1} \sum_{j \neq i} \left(S_{ij} - \frac{1}{1 + 10^{\{(R_j - R_i)/400\}}} \right)$$

S vale 1 si i llegó antes que j y 0 si llegó después. K es 48 en las primeras 10 carreras de la temporada y 32 después. Rating inicial: 1000. El rating se guarda en `ranked/s{N}` y se copia a la leaderboard de la temporada.

División	Rating
Bronce	Menos de 1100
Plata	1100 a 1299
Oro	1300 a 1499
Platino	1500 a 1699
Diamante	1700 a 1899
Leyenda	1900 o más

Temporadas

- Las fechas de cada temporada viven en el catálogo `seasons`. El pase de temporada usa las mismas fechas.
- **Cierre perezoso.** La primera vez que un jugador entra después del cierre, el servidor lee su división más alta y su puesto final en la tabla de la temporada anterior, le envía los premios al inbox y crea su `ranked/s{N+1}`.
- **Reinicio parcial.** $\text{Rating nuevo} = 1000 + (\text{rating anterior} - 1000) \times 0,5$.
- **Premios.** Por división más alta alcanzada (gemas y un cosmético de temporada) y un extra para el top 100.

Torneos

vida de un torneo · 6 pasos

Formato	Cómo se puntúa	Duración típica	Configuración en Nakama
Contrarreloj	Mejor tiempo en una pista fija	Viernes a domingo	Operador <code>best</code> , orden ascendente, máximo 20 intentos
Copa	Puntos por posición en hasta 10 carreras (10, 8, 6, 5, 4, 3 en carreras de 6 autos)	48 horas	Operador <code>incr</code> , orden descendente, máximo 10 intentos
Copa de clubes	Suma de puntos de los miembros	Una semana	Tabla con el club como dueño

- Unirse es obligatorio (`tournament_join`) y puede costar monedas. La Copa tiene su propio `mode` en el matchmaker.
- Al terminar, el callback de fin de torneo reparte premios por tramos de puesto a través del inbox.
- Mientras la red sea relay puro, los premios del top 10 se retienen 24 h para revisar resultados marcados. El contrarreloj guarda los parciales por checkpoint del mejor intento de cada jugador para esa revisión.

Leaderboards

La leaderboard actual, escrita por el cliente, se descarta y se reconstruye con tres reglas: todas las tablas son autoritativas (solo escribe el servidor), se crean desde un registro en el catálogo y se alimentan únicamente del evento `RaceCompleted`. El cliente deja de enviar puntajes.

leaderboards · 1 entrada, 6 tablas, 4 vistas

Tablas

ID	Qué mide	Operador	Orden	Reinicio
<code>tt_{pista}_{clase}_all</code>	Mejor tiempo de carrera por pista y clase	<code>best</code>	Ascendente	Nunca
<code>tt_{pista}_{clase}_week</code>	Mejor tiempo de la semana	<code>best</code>	Ascendente	Lunes 00:00 UTC
<code>lap_{pista}_{clase}_all</code>	Mejor vuelta	<code>best</code>	Ascendente	Nunca
<code>ranked_s{N}</code>	Rating de la temporada N	<code>set</code>	Descendente	Tabla nueva por temporada
<code>wins_week</code>	Victorias en carrera rápida y ranked	<code>incr</code>	Descendente	Lunes 00:00 UTC
<code>club_week</code>	Puntos del club (dueño: el grupo)	<code>incr</code>	Descendente	Lunes 00:00 UTC

Los tiempos se guardan en milisegundos como puntaje entero. El subpuntaje guarda la marca de tiempo del registro para desempatar a favor de quien lo logró primero.

Reglas de escritura

- Solo entran resultados con confianza `quorum` o `server`. Un resultado `client` entra a las tablas de tiempo solo si viene de contrarreloj y pasó todas las validaciones; queda marcado en su metadata.

- Cada registro guarda en metadata el auto, la plataforma, el control, la versión del cliente y el ID de sesión. Con eso se puede filtrar, auditar y borrar.
- Un jugador marcado por trampas se retira de todas las tablas con una sola función del módulo.

Vistas para el cliente

`lb_get` recibe el ID de tabla y la vista, y devuelve siempre la misma forma de respuesta:

- **Global:** top 50 paginado.
- **Alrededor de mí:** 10 puestos arriba y abajo del jugador.
- **Amigos:** registros de la lista de amigos más el jugador.
- **Club:** registros de los miembros del club.

La respuesta se enriquece en el servidor con nombre, avatar, división y club de cada fila, para que el cliente no haga consultas extra.

Reemplazo de la tabla actual

1. Crear las tablas nuevas desde el registro al arrancar el servidor.
2. Borrar la tabla actual y sus datos; no se migran.
3. Quitar del cliente la escritura directa y bloquearla en el servidor con un hook `before` que rechace escrituras de cliente.

Misiones diarias y pase de temporada

Las misiones dan el motivo para entrar cada día y el pase convierte ese hábito en una meta de 6 semanas. Ambos avanzan solo con eventos `RaceCompleted`, así que el cliente no puede reportar progreso.

misiones y pase · 8 pasos

Misiones

- **Diarias:** 3 por día, elegidas del catálogo con una semilla formada por el ID del jugador y la fecha UTC. Un cambio gratis por día.
- **Semanales:** 3 por semana, más largas, con mejor premio.
- **Definición por datos.** Cada misión del catálogo es un contador con filtros: `{evento: carrera_terminada, filtro: {modo, pista, clase, posiciónMáxima}, meta: 5}`. Añadir misiones es editar JSON.

- **Asignación perezosa.** `missions_get` compara el sello de fecha guardado con el día actual. Si cambió, reemplaza las misiones; lo no reclamado se pierde.
- **Reclamo.** `mission_claim` verifica que esté completa y no reclamada, y paga monedas y XP de pase.

Ejemplos para el catálogo inicial: terminar 3 carreras, quedar en el podio 2 veces, ganar 1 carrera ranked, correr en 3 pistas distintas, completar 1 contrarreloj, correr 2 carreras con un auto clase C.

Pase de temporada

- 40 niveles, dos carriles (gratis y premium), misma duración que la temporada ranked.
- La XP de pase sale de misiones (fuente principal) y de cada carrera terminada (fuente menor). Con juego diario normal, el nivel 40 se alcanza hacia la semana 5.
- El premium se compra con gemas. Al comprarlo se pueden reclamar de golpe los niveles premium ya alcanzados.
- `pass_claim` recibe nivel y carril, es idempotente y entrega monedas, gemas, cosméticos o autos según el catálogo.
- Al cerrar la temporada, lo alcanzado y no reclamado se envía al inbox en el cierre perezoso.

Logros

Metas permanentes de una sola vez (100 carreras, primer Diamante, todos los autos de una clase). Usan el mismo motor de contadores que las misiones, sin fecha de vencimiento. Se dejan para la misma fase porque el costo adicional es mínimo.

Amigos, clubes y chat

Casi todo lo social ya existe en Nakama; el trabajo es de cliente, reglas y moderación. El orden recomendado es amigos, luego grupos de juego, luego clubes y por último chat.

social · 4 piezas sobre lo nativo de Nakama

Amigos

- Lista de amigos, solicitudes y bloqueo: nativo. Búsqueda por nombre de usuario o código de amigo.
- **Presencia.** El cliente sigue el estado de sus amigos y publica el propio: en menú, en cola, en carrera, en sala privada con cupo.

- **Invitaciones.** Invitar a un amigo a un grupo o a una sala privada envía una notificación en tiempo real; si está desconectado, queda guardada.
- **Recientes.** `RaceCompleted` guarda los últimos 20 rivales humanos de cada jugador para poder agregarlos después de una carrera.

Grupos de juego

Parties de Nakama: un líder, hasta 4 miembros, cola conjunta en el matchmaker. Viven solo mientras los miembros están conectados.

Clubes

- Son los groups de Nakama: hasta 30 miembros, abiertos o por solicitud, con los roles nativos (líder, administrador, miembro).
- La metadata del grupo guarda emblema, lema, región y requisito mínimo de división. Solo la cambia el servidor.
- **Puntos de club.** Cada carrera terminada por un miembro suma a `club_week` : 3 puntos por victoria, 2 por podio, 1 por terminar. La tabla se reinicia cada lunes y los 3 mejores miembros del club ganador reciben un extra.
- Crear un club cuesta monedas, para frenar la creación masiva.

Chat

Canal	Disponible	Notas
Club	Con clubes	Persistente, historial de 7 días
Directo entre amigos	Con amigos	Solo entre amigos mutuos
Sala y carrera	Con salas	Frases rápidas predefinidas, sin texto libre
Global	No se recomienda	Costo de moderación alto para el valor que da

Moderación

- Un hook `before` sobre el envío de mensajes aplica filtro de palabras y límite de frecuencia (1 mensaje por segundo, 20 por minuto).
- `report_player` guarda el reporte con contexto (sesión, canal, últimos mensajes). Tres reportes de jugadores distintos en 24 h silencian el chat de forma automática y temporal.
- Los nombres de jugador y de club pasan por el mismo filtro al crearse o cambiarse.

- Revisar los requisitos de cada tienda para chat con texto libre (clasificación por edad, bloqueo y reporte visibles) antes de activarlo.

Tienda, compras y cuentas multiplataforma

Una sola cuenta de jugador con varios métodos de acceso enlazados es lo que hace real el multiplataforma: el progreso sigue al jugador y las compras de cualquier tienda llegan al mismo wallet.

Cuentas

cuentas · de invitado a multiplataforma

- **Entrada sin fricción.** En móvil, cuenta de invitado por ID de dispositivo. En PC, la cuenta de la tienda (Steam u otra).
- **Enlace.** Desde ajustes, el jugador enlaza Google, Apple, Steam o correo a su cuenta. Con un método enlazado, puede entrar desde otra plataforma y encontrar todo su progreso.
- **Conflicto de enlace.** Si el método ya pertenece a otra cuenta, el cliente muestra ambas (nivel, autos, división) y el jugador elige con cuál quedarse. Nunca se fusionan cuentas.
- **Recordatorio.** A partir del nivel 5, una cuenta de invitado sin enlazar ve un aviso con un premio pequeño por enlazar. Una cuenta de invitado se pierde al desinstalar.
- **Borrado.** RPC `account_delete` para cumplir el requisito de las tiendas de poder eliminar la cuenta desde la app.

Tienda dentro del juego

- El catálogo `store` define secciones: autos por clase, cosméticos, oferta diaria rotativa (misma semilla diaria que las misiones) y paquetes.
- `store_get` devuelve solo lo que ese jugador puede ver: filtra por nivel, por lo que ya posee y por eventos activos.
- `store_buy` recibe el ID de la oferta, verifica vigencia y requisitos, descuenta y entrega en una transacción.

Compras con dinero real

El juego vende con dinero real en móvil y PC. Cada producto es una lista de premios en el catálogo `iap_products`, así que crear un paquete nuevo es editar datos, no código.

Producto	Contenido	Tipo
Paquetes de gemas y de monedas	Moneda en varios tamaños	Consumible
Paquetes de skins y colores	Cosméticos de un tema o de un auto	Permanente
Autos	Un auto, con o sin skin exclusiva	Permanente
Personalizaciones	Estelas, bocinas, llantas, calcomanías	Permanente
Pase de temporada	Carril premium de la temporada actual	Permanente por temporada
Paquetes de evento y especiales	Mezcla de moneda, auto y cosméticos, con vigencia y límite de compras	Mixto

Los productos permanentes se pueden restaurar en un dispositivo nuevo; los consumibles no.

compra con dinero real · 6 pasos, 3 actores

- Nakama valida de forma nativa los recibos de Apple, Google y Huawei, y registra cada compra para detectar recibos repetidos. La tienda de PC que se elija requiere integración propia contra su API.
- Flujo: el cliente compra con el SDK de la tienda, envía el recibo a `iap_validate`, el servidor valida, entrega las gemas con el ID de transacción como clave de idempotencia y recién entonces el cliente confirma la compra a la tienda.
- Los reembolsos notificados por las tiendas descuentan lo entregado; si el saldo queda negativo, la cuenta se marca.
- Cada entrega de gemas registra la plataforma de origen. Algunas plataformas restringen usar en ellas moneda comprada en otra; tener el dato permite cumplirlo sin rehacer el wallet.

Anuncios con recompensa

Si se usan, el premio lo entrega el servidor al recibir la verificación de servidor a servidor de la red de anuncios, no cuando el cliente dice que el video terminó. Tope diario por jugador en el catálogo.

Anti-trampas por etapas

Con relay puro no se puede impedir que un cliente modificado mienta; sí se puede lograr que mentir no pague. Cada etapa se activa cuando la anterior deja de alcanzar, y ninguna obliga a tocar el metajuego porque todas terminan en el mismo `RaceCompleted`.

Etapa	Qué añade	Qué frena	Cuándo
0. Higiene	Solo el servidor escribe. Roster, reloj del servidor, tiempo mínimo plausible, quórum, idempotencia, límite de llamadas por RPC	Puntajes inventados, premios duplicados, tiempos imposibles	Fase 1, obligatoria
1. Telemetría	Parciales por checkpoint en cada resultado, detección estadística (mejoras bruscas, parciales imposibles entre checkpoints), marcas ocultas, reportes de jugadores, revisión manual del top	Tiempos falsos pero plausibles, tramposos reincidentes	Antes del primer torneo con premio
2. Relay observado	El match pasa a un handler autoritativo delgado: reenvía los mensajes igual que hoy, pero ve los cruces de checkpoint, los sella con su reloj y decide el orden de llegada	Resultados falsos en carreras con rivales, host deshonesto	Si el ranked sufre trampas visibles
3. Verificación de física	Repetición de carreras sospechosas en un Unity sin gráficos a partir de las entradas grabadas	Velocidad y física alteradas	Solo si el juego crece lo suficiente para justificarlo

Etapa 2 en detalle

Es el paso natural desde relay puro y cuesta poco. El hook `matchmakerMatched` pasa a crear un match autoritativo y devolver su ID; el cliente se une igual que hoy. El handler no simula física: recibe cada mensaje, lo reenvía a los demás y solo interpreta dos códigos de operación, cruce de checkpoint y meta. Con eso el servidor produce resultados con confianza `server`. Para dejarlo preparado desde ya, conviene que el protocolo de carrera actual envíe checkpoint y meta como mensajes con código propio, separados del estado del auto.

relay puro frente a relay observado

Sanciones

1. **Marca oculta.** El jugador sigue jugando, pero sus resultados no entran a tablas ni torneos.
2. **Retiro de tablas y premios retenidos.**
3. **Suspensión de ranked y torneos** por tiempo definido.
4. **Baneo de cuenta** desde la consola de Nakama.

Toda sanción automática llega como máximo al nivel 1. Del 2 en adelante decide una persona.

LiveOps, analítica y operación

El servidor actual en Docker sirve para desarrollar; antes de abrir el juego a jugadores reales necesita tres entornos, copias de seguridad, monitoreo y una prueba de carga. LiveOps y analítica se montan sobre lo que ya existe.

LiveOps

configuración remota · 4 pasos

- **Configuración remota.** `liveops/config` guarda flags y valores ajustables (multiplicadores de premio, tiempos de matchmaking, segmentación). Se edita desde la consola de Nakama y llega al cliente por `config_get`.
- **Calendario de eventos.** Lista con fecha de inicio y fin: XP doble, pista destacada, torneo activo, oferta especial. Cada módulo consulta si hay un evento que lo afecte.

- **Inbox.** Notificaciones persistentes con premio adjunto y fecha de vencimiento; `inbox_claim` las cobra. Lo usan temporadas, torneos, compensaciones y regalos.
- **Mantenimiento y versión mínima.** Un flag bloquea el acceso con un mensaje; otro obliga a actualizar el cliente.
- **Herramientas de soporte.** RPCs protegidos con la clave HTTP del servidor para dar o quitar objetos, enviar correo masivo y aplicar sanciones.

Analítica

Los eventos de negocio se emiten desde el servidor, donde no se pueden falsear: sesión creada, carrera completada, movimiento de wallet con motivo, misión reclamada, nivel de pase, compra, emparejamiento (tiempo en cola, humanos y bots). El cliente emite solo eventos de interfaz y rendimiento. Indicadores a seguir desde el primer día: retención a 1, 7 y 30 días, carreras por sesión, tiempo en cola, porcentaje de carreras con bots, monedas ganadas contra gastadas por día y porcentaje de resultados marcados.

Operación

- ☐ Tres entornos separados: local, staging y producción, cada uno con su base de datos
- ☐ Cambiar todas las claves por defecto: server key, HTTP key, clave de firma de sesión, usuario y contraseña de la consola
- ☐ TLS delante de Nakama con un proxy inverso; consola accesible solo por VPN o lista de IPs
- ☐ PostgreSQL administrado o con copias automáticas diarias y una restauración probada
- ☐ El código TypeScript se compila y se empaqueta en la imagen Docker desde un pipeline; nada se edita a mano en el servidor
- ☐ Métricas de Nakama exportadas a Prometheus con paneles y alertas: sesiones, latencia de RPC, errores, CPU, memoria, conexiones a la base
- ☐ Logs centralizados con el ID de sesión de carrera en cada línea relevante
- ☐ Prueba de carga con clientes sin gráficos: 500 jugadores simultáneos en carreras de 6 durante 1 hora
- ☐ Procedimiento escrito de despliegue y de vuelta atrás
- ☐ Límite de frecuencia por RPC y por jugador

Regiones y escalado

La versión de código abierto de Nakama corre en un solo nodo; el clúster es parte de la versión Enterprise y de Heroic Cloud. Para cubrir América y Europa sin dividir a los jugadores, el diseño separa dos papeles:

- **Servidor principal** (costa este de Estados Unidos): cuentas, wallet, garaje, tablas, misiones, social y todos los RPC. Es la única fuente de datos.
- **Nodo de relay por región** (América y Europa): solo cola de matchmaking y carrera en tiempo real. El jugador entra con un token firmado de corta duración que emite el principal. Al emparejar, el nodo le pide al principal que cree la sesión de carrera.

Un jugador europeo corre con baja latencia contra otros europeos y su progreso vive en el mismo lugar que el de todos. Los RPC de metajuego toleran bien 100 a 150 ms de latencia. Este esquema necesita una prueba de concepto, prevista en la Fase 5. Si no resulta, la alternativa es un despliegue independiente por región, con cuentas y tablas separadas.

despliegue · 1 principal, 2 relays, 1 base

Un nodo bien dimensionado alcanza para un lanzamiento, porque el relay consume poco. La prueba de carga dice cuál es el techo real; a partir de ahí se escala en vertical o se añaden nodos de relay.

Hoja de ruta

hoja de ruta · 9 fases, 1 línea de corte

Las fases 1 a 5 van en ese orden porque cada una usa la anterior. Las fases 6 a 9 se pueden reordenar según lo que pidan los jugadores después del lanzamiento en pruebas; la Fase 9 puede adelantarse si se quiere medir ingresos desde las pruebas. No hay fechas todavía. Las tareas, pruebas y criterio de terminado de cada fase están en

Checklist de desarrollo por fases — Juego de carreras con Nakama .

Riesgos y decisiones abiertas

El riesgo mayor es de alcance: cuatro pilares de metajuego son mucho para un equipo de dos personas. La hoja de ruta lo trata con una línea de corte: al terminar la Fase 5 el juego ya se puede lanzar en pruebas, y el resto entra como actualizaciones.

Riesgos

Riesgo	Efecto	Mitigación
Trampas en tablas de tiempo con relay puro	Tablas y torneos pierden valor	Etapas 0 y 1 de anti-trampas, premios retenidos, paso a relay observado
Host con mala conexión o deshonesto	Carreras injustas, bots manipulados	Host por menor latencia, sucesión, quórum de resultados
Economía desbalanceada al lanzar	Progreso demasiado rápido o demasiado lento	Hoja de cálculo previa, valores en configuración remota, ledger con motivos
Nodo único de Nakama	Una caída detiene todo el juego	Copias probadas, monitoreo, prueba de carga, vuelta atrás escrita
Dos regiones con un Nakama de un solo nodo	Con despliegues independientes, cuentas, amigos y tablas quedarían separados por región	Servidor principal único y nodo de relay por región; prueba de concepto en la Fase 5
Chat con texto libre	Costo de moderación y requisitos de tiendas	Frases rápidas primero; texto libre solo en club y directo
Poca gente en cola al inicio	Esperas largas, ranked vacío	Bots en carrera rápida, ranked que arranca con 2, 4 o 6 humanos, pool única por región

Decisiones tomadas

Tema	Decisión
Estadísticas en ranked	Igualadas al tope de la clase
Modelo de negocio	Venta con dinero real: monedas y gemas, skins y colores, autos, personalizaciones, pase de temporada, paquetes de evento y especiales
Metajuego	Se desarrolla en casa; no se licencia una librería comercial

Tema	Decisión
Plataformas	Móviles y PC
Regiones	América y Europa
Leaderboard actual	Se descarta y se reconstruye
Tamaño de carrera	2, 4 y 6 autos
Contenido de lanzamiento	Al menos 6 pistas y 6 autos
Equipo	1 persona en servidor, 1 en cliente

Decisiones por tomar

- ☐ **Tienda de PC:** define el método de acceso y el sistema de compras a integrar. Antes de la Fase 5.
- ☐ **Regiones:** confirmar el esquema de servidor principal y relay regional con la prueba de concepto, o pasar a despliegues independientes. En la Fase 5.
- ☐ **Tamaño en ranked:** una sola cola que forma carreras de 2, 4 o 6 humanos (propuesto) o colas separadas por tamaño. Antes de la Fase 4.
- ☐ **Destino de la analítica.** Antes de la Fase 5.
- ☐ **Fecha objetivo** del lanzamiento en pruebas; con ella se ponen fechas a las fases.

Fuentes

- [Client Relayed Multiplayer — documentación de Nakama](#): qué conserva el servidor en un match por relay.
- [Authoritative Multiplayer — documentación de Nakama](#): match handlers y el hook de emparejamiento.
- [In-app Purchase Validation — documentación de Nakama](#): tiendas con validación nativa.

Los valores numéricos del plan (premios, rangos de rating, tiempos de espera, tamaños) son propuestas de partida, no cifras de la documentación.